CS122A Final Project
An FPGA Software-Defined Radio Receiver

Spectrum, Weather-Satellite, and Aircraft Display on an ECP5

Marco Miralles
Gage Shaddock

Terminology

- SDR - Software Defined Radio
- RF - Radio Frequency
- IQ - In-phase and Quadrature samples
- LO - Local Oscillator
- FPGA - Field Programmable Gate Array
- FFT - Fast Fourier Transform
- RDS - Radio Data System (the digital sub-carrier on FM broadcast)
- PS name - Program Service name (the 8-character station label in RDS)
- MPX - Multiplex (the composite FM baseband)
- GOES - Geostationary Operational Environmental Satellite
- APT - Automatic Picture Transmission (NOAA weather-image format)
- ADS-B - Automatic Dependent Surveillance Broadcast (aircraft position beacon)
- LCD - Liquid Crystal Display
- DAC - Digital to Analog Converter
- SDRAM - Synchronous Dynamic Random-Access Memory
- TLV - Type-Length-Value (the framing for the Pluto link)
- SPI - Serial Peripheral Interface
- I^2C - Inter-Integrated Circuit
- I^2S - Inter-Integrated Circuit Sound
- MCU - Microcontroller Unit

Table of Contents

Introduction

The goal of this project was to build an SDR whose output included both audio and visual components, where a RF front end was used to drive a speaker and an LCD screen. The initial idea was to have support for multiple processing modes that produced different visuals. In FM mode the screen shows a live spectrum and a scrolling waterfall while audio plays out of a speaker. In GOES mode it paints a weather-satellite image line by line as it arrives. In ADS-B mode it draws a map of the local airspace with aircraft plotted as dots wherever they broadcast their positions. The user chooses a mode by touch, the radio retunes itself, and the screen changes to match.

A PlutoSky 7020 pairs an Analog Devices AD9363 transceiver with a Xilinx Zynq SoC. The project starts at the Zynq chip thag runs a custom Vivado block design that adds an AXI Quad SPI peripheral, wired through the programmable logic to the JP5 expansion header. A custom userspace application on the ARM core uses libiio to configure the AD9363 and capture IQ samples, pack them into TLV frames, and stream them out over that SPI link to the iCESugar-Pro at about 12.5 MHz. This gave us a clean, self-contained data pipe out of the radio and let us focus the rest of the project on signal processing and display.

The design is split across three boards, each doing what it is best at. The PlutoSky is the RF front end. It tunes, samples, and streams IQ. The iCESugar-Pro, a Lattice ECP5 FPGA with 32 MB of SDRAM, is the hub of the system. It ingests the IQ stream, runs all of the digital signal processing, composes the screen, drives the LCD, and feeds an audio DAC. The Raspberry Pi Pico 2W owns the user interface. It reads the capacitive touch panel and sends compact UI-state frames to the FPGA, which in turn relays tuning commands back to the radio. Only one mode runs at a time, because the AD9363 cannot hold two receive frequencies at once, but switching between them is meant to feel seamless.

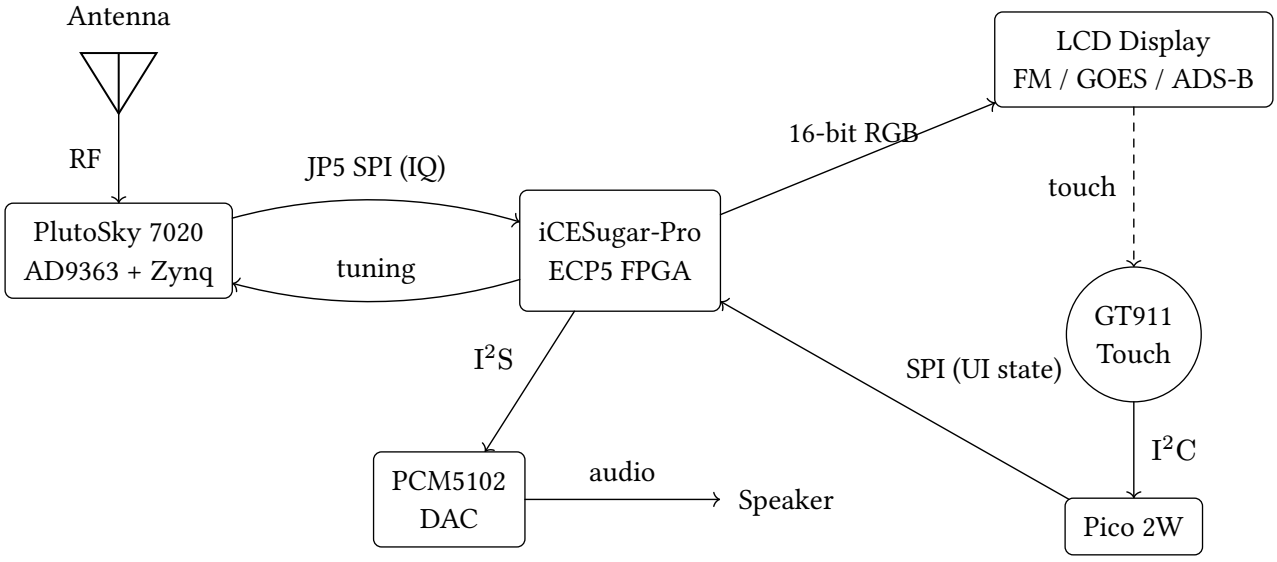

Figure 1: System block diagram. The ECP5 is the hub of the design: it ingests IQ samples from the PlutoSky, renders the display directly in fabric, drives the audio DAC, and relays tuning commands back to the radio. The FM path (spectrum, waterfall, and audio) is proven end to end; GOES and ADS-B are planned modes fed from the same front end.

The mode that works end to end, on hardware, is FM. Tuned to a broadcast station, the PlutoSky streams raw IQ to the ECP5, which demodulates it, plays the audio out of the speaker, and at the same time runs the IQ through a 256-point FFT to draw the spectrum and waterfall. The two image modes, GOES and ADS-B, are decoded on the PlutoSky and their results already cross the link to the FPGA, but the rendering that would paint a weather image or plot aircraft on the map is not yet

wired, so today those packets are received and counted rather than shown. The body of this report covers how the working parts fit together and where the project stops short.

RF front end	PlutoSky 7020 (AD9363 transceiver + XC7Z020 Zynq)
Display FPGA	iCESugar-Pro (Lattice ECP5-25K) + 32 MB SDRAM
UI microcontroller	Raspberry Pi Pico 2W (RP2350)
Display	4.3″ 800×480 RGB-parallel LCD, 60 Hz, RGB565
Touch	Goodix GT911 capacitive controller over I^2C
Audio DAC	Texas Instruments PCM5102 over I^2S

Table 1: The six hardware components of the receiver.

Elements of Complexity

The course had already walked us through building a display controller that drives the 800×480 RGB LCD from a framebuffer, so the raw video output stage was a known quantity we built on top of. The original and difficult work was on the radio and memory side, and in stitching three boards into one real-time system. The pieces that carried the most complexity were these:

- **A streaming 256-point FFT in fabric.** `fft256` is built from a `butterfly` datapath and a `twiddle_rom`, running continuously on the incoming IQ to produce one spectrum row per frame.
- **FM demodulation in hardware.** `fm_demod` recovers audio from the complex baseband using the cross-product discriminator: two multiplies and a subtract approximate the instantaneous phase difference without a division or arctangent.
- **A from-scratch SDRAM controller and arbiter.** `sdram_ctrl` and `sdram_arb` bring up the 32 MB SDRAM and share it between three clients on a fixed priority. Getting it reliable meant diagnosing a hardware-level page-boundary fault on the part itself, described later.
- **Hard real-time scan-out.** `scan_out` and `line_cache` read one display line ahead of the beam and cross from the 100 MHz memory clock to the 30 MHz pixel clock without tearing.
- **The waterfall compositor.** `compositor` scrolls the history by moving a base-row pointer instead of copying memory, so a new row costs one short write.
- **A custom Zynq block design.** Our Vivado block design adapts maia-hdl for the correct PS7 configuration, adds the AXI Quad SPI IP for the JP5 link, and packages the result into BOOT.BIN.
- **A custom capacitive touch driver.** The GT911 controller was not provided. We wrote the I^2C driver and the touch handling on the Pico.
- **Multi-device integration.** Three boards talk over four different links using a custom CRC-checked wire protocol, with the ECP5 acting as the hub and relaying tuning commands back to the radio.
- **A C to FPGA verification harness.** The render path has a portable C reference and a Verilator parity test that proves the SystemVerilog produces pixel-identical output, which let us develop and check the display logic away from hardware.
- **DSP zoom.** `spectrum_zoom_decimator` changes the FFT input rate to zoom the spectrum while keeping all 256 bins on screen.

Signal Processing Background

IQ Sampling

The AD9363 chip inside the PlutoSky does all the analog RF work of the project. We tune it to a center frequency and it hands back a stream of complex samples, each one a pair (I, Q) describing the signal's amplitude and phase relative to the local oscillator. Mathematically the radio gives us the complex baseband

$$z[n] = I[n] + j\,Q[n]$$

already downconverted and decimated to a manageable rate. Everything our design does begins from that stream.

FM Demodulation

In frequency modulation the information is contained in the instantaneous frequency of the signal, and frequency is simply the rate of change of phase. With complex samples the phase is $\varphi[n] = \operatorname{atan2}(Q[n], I[n])$, so the demodulated output is the phase difference between consecutive samples. The ideal discriminator multiplies each sample by the conjugate of the previous one and takes the angle:

$$m[n] \propto \arg(z[n] \cdot z^*[n-1]) = \operatorname{atan2}(I[n-1]Q[n] - I[n]Q[n-1],\, I[n]I[n-1] + Q[n]Q[n-1])$$

The numerator is the cross product and the denominator is the dot product of consecutive IQ vectors. The `fm_demod` module drops the dot product and arctangent, using only the cross product as the discriminator output:

$$m[n] \propto I[n-1]Q[n] - I[n]Q[n-1]$$

This is two multiplies and a subtract. It is an approximation of the full atan2, valid when signal amplitude is roughly constant, which it is after the AD9363′s automatic gain control.

Spectrum and Waterfall

The same IQ stream that feeds the demodulator also feeds a 256-point FFT (`fft256`, built from a `butterfly` datapath and a `twiddle_rom`). Taking the magnitude of each bin gives the power at that frequency, and mapping magnitude through a color palette gives one row of pixels. Stacked over time, those rows are the waterfall. The vertical axis is time, the horizontal axis is frequency, and brightness is signal strength, so a steady carrier draws a bright vertical streak and a transient flashes and scrolls away. The instantaneous trace across the top is the spectrum. Both are derived from the same FFT output. The spectrum is the newest row drawn as a line, and the waterfall is its history.

RDS

FM broadcast carries a low-rate digital channel, the Radio Data System, on a sub-carrier at 57 kHz within the multiplex, which is why RDS recovery starts from the FM demodulator's output rather than from raw IQ. The data is differentially encoded BPSK at 1187.5 bits per second, grouped into 26-bit blocks with offset words that let a receiver find block and group boundaries without a separate sync channel. Our gateware recovers it in three stages. `rds_demod` recovers the bitstream from the multiplex, `rds_sync` locks to the block and group structure, and `rds_group` assembles the groups and extracts the 8-character Program Service name, the short station label a car radio shows. The decode chain works in simulation, recovering the station name from captured multiplex, but we were not able to get it to lock reliably on live hardware, so RDS is not part of the working hardware feature set. When no name has been decoded, the status row falls back to text supplied by the Pico.

System Design

The PlutoSky Front End

The PlutoSky 7020 pairs the Analog Devices AD9363 RF transceiver with a Xilinx XC7Z020 Zynq SoC in a CLG400 package. The starting point for the programmable logic was the maia-hdl `pluto_iio` block design, which provides the `axi_ad9361` RF interface, ADC DMA, and the maia-sdr spectrometer IP. An AXI Quad SPI instance was added to the design and mapped into the AXI GP0 address space so the ARM core can reach it through a memory-mapped register interface. This design is built using `system_bd.tcl` script.

On top of the block design, `system_top.v` adds the physical layer: IOBUF instantiations for the AD9363 control and explicit routing of the JP5 SPI signals from the AXI Quad SPI IP to Bank 13 output pins. The bitstream, First Stage Bootloader, and U-Boot are packed into a single BOOT.BIN image, so the programmable logic is configured and the JP5 outputs are driven before the Linux kernel starts.

The ARM userspace runtime uses libiio to configure the AD9363: center frequency, a 2.6042 MHz ADC sample rate, and receive gain. IQ samples arrive in a kernel DMA buffer and a software decimation step of ten reduces the stream to 260,417 complex samples per second, the rate the JP5 link is sized for. The runtime packs those samples into TLV_IQ frames and writes them through the AXI SPI registers to the JP5 header, driving the iCeSugar as a continuous SPI master at about 12.5 MHz. Between outgoing bursts the runtime drains the SPI receive FIFO to pick up command bytes the ECP5 has queued on the MISO line. A SET_FREQ command calls back into libiio to retune the AD9363 center frequency, completing the control loop that starts at the touchscreen.

The Radio Pipeline

Everything past the radio happens in the ECP5. Bytes arrive on the JP5 SPI link, `spi_frame_rx` reassembles them, and `tlv_demux` routes each packet by type. IQ packets land in `tlv_iq_sink`, which feeds the three branches shown in the figure below: the FFT branch for the spectrum and waterfall, the `fm_demod` branch for audio out through an `audio_fifo` and `i2s_tx` to the PCM5102, and the RDS branch off the recovered multiplex. This pipeline, and the SDRAM controller that backs it, is the heart of the project's original work.

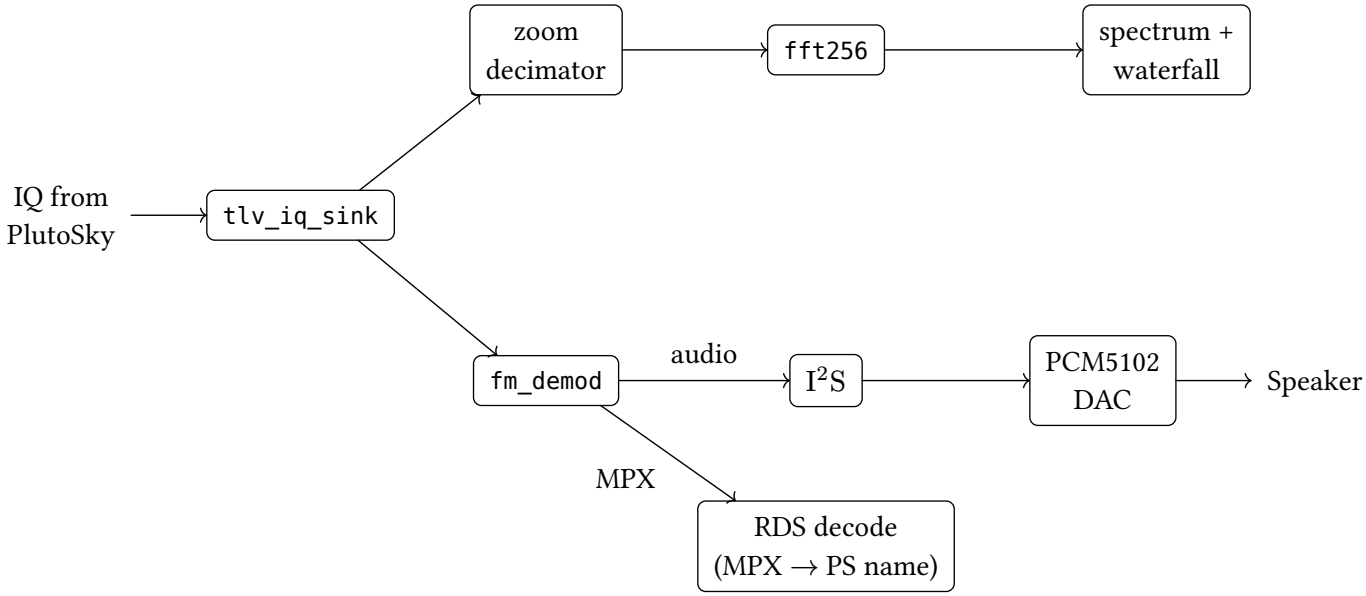

Figure 5: The FM signal-processing chain inside the ECP5. One IQ stream splits three ways: an FFT branch that feeds the spectrum and waterfall, an audio branch through the FM demodulator to the DAC, and an RDS branch that recovers the station name from the FM multiplex (verified in simulation; not instantiated in the final hardware build). The spectrum and waterfall outputs are written to SDRAM by the compositor in the display pipeline.

The Display Path

The display controller that turns a framebuffer into RGB and sync signals for the LCD came out of the course, so we treated the video output stage as a building block. Our work on the display side was making that framebuffer carry live radio data in real time, which is built around the iCESugar-Pro's 32 MB SDRAM. The framebuffer, the waterfall history, and the GOES and ADS-B image regions all live in SDRAM, and three clients share it through a fixed-priority arbiter (`sdram_arb`). The highest priority is the scan-out reader, because missing a line shows up immediately as a tear. Below it sit the compositor's writes and the ingest of new IQ.

A `scan_timing` module owns the 800×480 at 60 Hz LCD timing and runs in the pixel-clock domain at about 30 MHz, while the SDRAM, arbiter, and compositor run at 100 MHz. The `line_cache` between them is a small on-chip buffer filled one line ahead of the beam. It is both the latency cushion that absorbs arbiter stalls and the crossing point between the two clock domains. The `compositor` advances the waterfall by one row not by moving a base-row pointer that the scan-out reader follows, so a new spectrum row costs a single short write. On top of the SDRAM-backed line, the `pixel_shader` draws the live overlay, which is the spectrum trace, the status bar, the frequency and band readout, the mode and volume indicators, and the touch crosshair, from font and sprite ROMs and the current UI state, emitting one RGB pixel per clock.

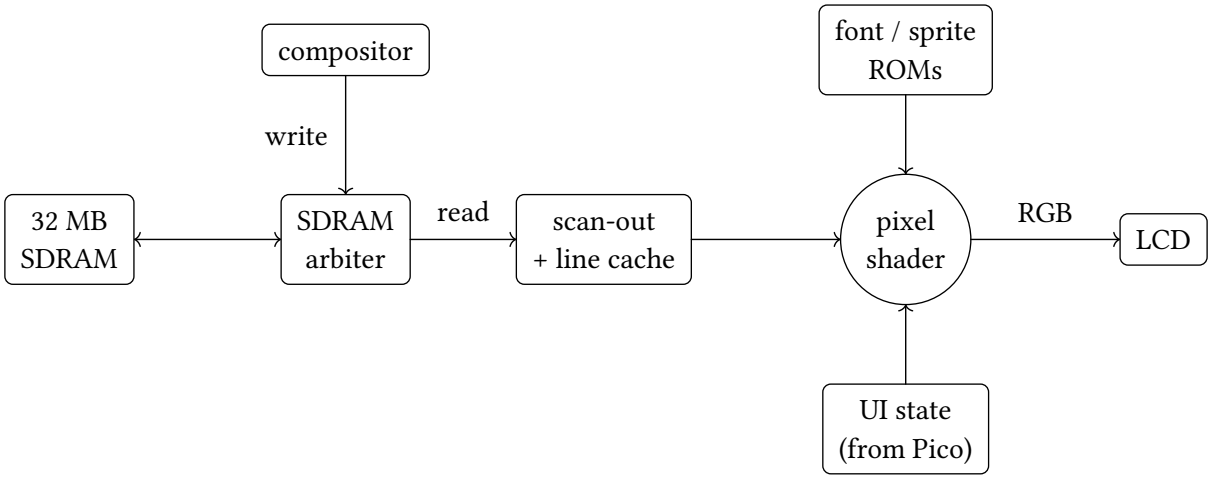

Figure 6: The display pipeline. The waterfall and image regions live in SDRAM; the compositor writes them and a scan-out reader pulls one line ahead into an on-chip cache. The pixel shader composes the final image on the fly, overlaying live UI elements from font and sprite ROMs and the current UI state on top of the SDRAM-backed line, and emits one RGB pixel per clock to the LCD.

The Touch Interface and Control Loop

The Pico 2W reads the Goodix GT911 capacitive panel over I^2C and keeps the authoritative copy of the user-facing state: mode, center frequency, volume, mute, and touch position. It sends that state to the FPGA as framed packets, each one a `0xA5` magic byte, an opcode, a little-endian length, a payload, and a CRC16-CCITT check, defined once in portable C and shared by both ends. On the FPGA two parsers read the same frames. `ui_wire_rx` keeps the fields the shader needs for the status bar, RDS line, touch overlay, and zoom, while `spi_ui_cmd_rx` extracts the fields that should become radio commands. Those commands, which set mode, frequency, volume, and mute, are applied locally by `control_regs` for audio and queued by `spi_backchannel` to be relayed back to the PlutoSky over the JP5 return line. The result is a closed loop. A touch on the glass changes the Pico's state, which retunes the radio.

User Guide

The receiver is meant to be used through the touchscreen alone. On power-up the three boards boot on their own and the radio comes up in FM mode, showing a live spectrum across the top of the screen with a waterfall scrolling beneath it. A status bar reports the current mode, the center frequency, the visible band, and the volume.

All interaction is through the touchscreen, handled by the Pico. A row of on-screen buttons along the side of the display drives the controls:

- **Tuning.** Touching the tuning control retunes the radio. The Pico sends the new frequency to the FPGA, the FPGA relays it to the PlutoSky, and both the spectrum and the audio follow.
- **Volume and mute.** One control sets the volume from 0 to 100, and a mute toggle silences the audio while leaving the display running.
- **Zoom.** The span control narrows or widens the visible band. The display always shows 256 bins, so zooming in raises the resolution in hertz per bin instead of changing the number of bars on screen.
- **Mode.** The mode button cycles FM, GOES, and ADS-B, and back to FM. Selecting a mode retunes the radio for that signal. FM is fully interactive today. The two image modes change the radio and the requested layout but do not yet paint their images.

Audio plays out of the speaker through the PCM5102 DAC, and any touch is echoed on screen as a crosshair so the user gets immediate feedback.

Hardware Components

Component	Part and role
RF front end	PlutoSky 7020: AD9363 transceiver with an XC7Z020 Zynq SoC, tunes and streams IQ
Display FPGA	iCESugar-Pro, Lattice ECP5-25K (CABGA256), with 32 MB IS42S16160B SDRAM
UI microcontroller	Raspberry Pi Pico 2W (RP2350)
LCD	4.3″ 800×480 RGB-parallel panel, 60 Hz, RGB565
Touch controller	Goodix GT911 capacitive controller (I^2C address 0x5D)
Audio DAC	Texas Instruments PCM5102 over I^2S
Speaker	Powered from the DAC's analog output
Antenna	Whip antenna at the PlutoSky RX port

Table 2: Hardware components used in the receiver.

Software Libraries and Tools

- **maia-sdr / maia-hdl** form the base firmware and FPGA design on the PlutoSky, providing the AD9363 interface and the IQ data path.
- **tezuka_fw** is the open-source Zynq firmware builder used to provide the pre-built FSBL, U-Boot, Linux kernel, device tree, and ramdisk for the PlutoSky. Our Vivado bitstream is combined with tezuka's bootloader components to produce the final BOOT.BIN.
- **libiio** on the PlutoSky reads IQ samples from the radio in userspace.
- **Raspberry Pi Pico SDK** builds the RP2350 firmware that drives the GT911 and the SPI link.
- **SDL2** renders the host harness on a laptop, used only for development.
- **CMake** was used to build the shared C, the host harness, and the test suite.
- **Verilator** simulates the SystemVerilog and runs the C to RTL parity tests.
- **Yosys**, **nextpnr-ecp5**, and **Project Trellis** synthesize and place the ECP5 bitstream.
- **Vivado** builds the PlutoSky bitstream, and the Vitis ARM cross-compiler builds its userspace code.
- Small **Python** scripts convert font, sprite, and palette data into the `.mem` ROMs the ECP5 loads.

Protocols

- **SPI**, used twice. The PlutoSky streams IQ to the ECP5 over the JP5 header (SPI mode 3), and the Pico sends UI-state frames to the ECP5 (SPI mode 0).
- **I^2C**, between the GT911 touch controller and the Pico.
- **I^2S**, carrying stereo audio from the ECP5 to the PCM5102 DAC.
- **AXI**, inside the Zynq, connecting the ARM cores to the AD9361 interface and the JP5 SPI block.
- **TLV framing** on the PlutoSky to ECP5 link, a type byte, a big-endian length, and a payload, used to carry IQ, image rows, and object lists over one stream.

Wiring Diagram

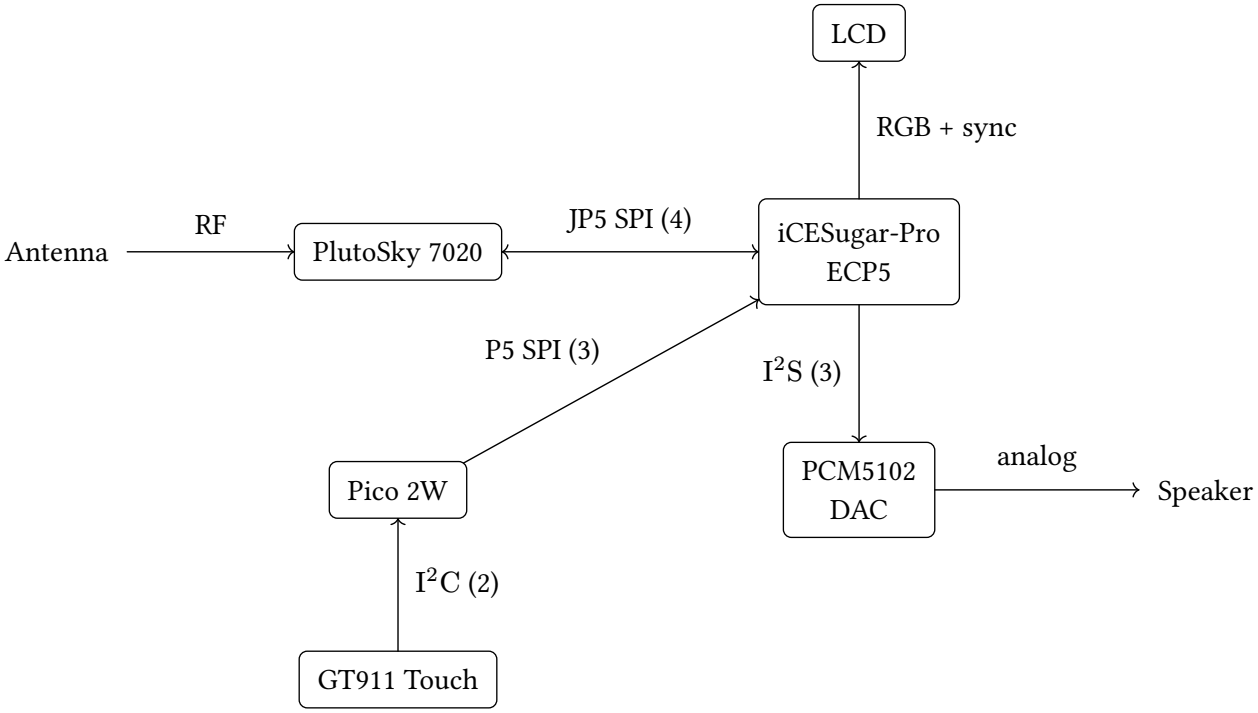

Figure 7: Physical wiring of the four boards. The label on each link gives the bus and its wire count; exact pin assignments are in the connection table.

The pin assignments for the three wired links are listed below, grouped by bus.

Bus	**Signal**	**From**	**To**
JP5 SPI (Pluto ↔ ECP5)	SCK	JP5 7 / Zynq V10	ECP5 D7
	MOSI	JP5 9 / Zynq U9	ECP5 D8
	CS	JP5 13 / Zynq T9	ECP5 D9
	MISO	ECP5 T3	JP5 11 / Zynq U10
Pico SPI (Pico → ECP5)	SCK	Pico GP18	ECP5 D12
	MOSI	Pico GP19	ECP5 C11
	CS	Pico GP17	ECP5 D13
Touch I²C (GT911 → Pico)	SDA	GT911 SDA	Pico GP4
	SCL	GT911 SCL	Pico GP5
I²S (ECP5 → PCM5102)	BCLK	ECP5 C3	PCM5102 BCK
	LRCLK	ECP5 R8	PCM5102 LRCK
	SDATA	ECP5 C4	PCM5102 DIN
	SCK	ECP5 E3	PCM5102 SCK

Table 3: Pin assignments for the inter-board links. The video (RGB) lines use the iCESugar-Pro header pinout from the course display controller. A common ground is shared across all boards.

Meeting the Proposal Requirements

Our proposal described an SDR that tunes FM or satellite-image radio, shows audio spectrum and waterfall visualizations on a touchscreen, plays audio through a speaker, and lets the user switch

modes and tune by touch, with the PlutoSky acquiring the radio signal, the Pico tracking state from touch input, and the ECP5 driving the display and audio. Measured against that proposal:

- **FM reception with spectrum and waterfall:** met. FM works end to end on hardware.
- **Audio output through the PCM5102 and a speaker:** met.
- **Touchscreen control of frequency, volume, and mode:** met. We added a zoom control beyond the proposal.
- **Switching modes by touch:** met for selection. FM renders fully, and the satellite-image and ADS-B modes select and retune but do not yet render their images.
- **Satellite-image mode:** partially met. The PlutoSky-side decoder and the transport to the ECP5 exist, but the FPGA-side image rendering is not wired.
- **ADS-B mode:** a stretch beyond the original FM-or-satellite proposal. Like satellite, the decoder and transport exist but the map rendering is not wired.

One architectural change from the proposal is worth noting. We had planned a direct Pico-to-PlutoSky UART for tuning. In the built system the Pico talks only to the ECP5, and the ECP5 relays tuning commands to the PlutoSky over the JP5 return line. Making the FPGA the single hub simplified the wiring and kept all control on one protocol.

Testing

Testing followed the signal chain from the radio front end through to the display and audio output, verifying each layer before connecting it to the next.

PlutoSky and the SPI Link

The PlutoSky was verified before any FPGA work depended on it. A C test program reads and writes the AXI Quad SPI register block to confirm the Vivado IP is correctly wired and reachable from the ARM core. On the host side, a Python script tunes the AD9363, captures IQ over SSH, and runs an FM demodulator to confirm the radio front end produces a clean baseband signal independently of the FPGA demodulation path.

ECP5 Module Simulations

Each functional block on the ECP5 has an Icarus Verilog testbench that runs without hardware. The FM discriminator is tested against known IQ inputs to verify the cross-product output. The FFT is checked for correct DC bin placement and complex-tone bin accuracy. The audio FIFO is exercised through fill, skip, and repeat conditions. The I^2S transmitter is checked for correct bit and frame timing. The zoom decimator is verified for output rate and sample alignment at each zoom level. The TLV receive chain is tested at each stage: the byte deserializer and start-of-frame marking, the packet demultiplexer routing by type byte, and the full chain end to end. The UI wire receiver is tested for correct frame parsing and field extraction. Finally, an integration simulation covers the SDRAM controller, arbiter, scan-out reader, line cache, and compositor together.

Hardware Diagnostic Bitstreams

Before the full build was assembled, each subsystem was verified on hardware with a minimal diagnostic bitstream that isolated one layer at a time. The SDRAM controller was tested first with a write/read pattern, with the LCD showing green on pass. The JP5 TLV link was then verified with an incrementing counter loopback that confirmed framing integrity end to end. The I^2S and DAC wiring were confirmed with a fixed sine tone requiring no SPI or FM demodulation. The FM demodulator was then exercised in isolation using an internally generated synthetic IQ source, with no PlutoSky connected. Finally, a minimal FM audio path ran live IQ from the PlutoSky through the demodulator to the DAC.

Bring-up followed this deliberate order so that each stage could be checked on its own: the SDRAM controller alone, then scan-out of a solid color, then the line cache and arbiter with a moving test pattern, then the Pico SPI link and a live touch cursor, then the font and sprite ROMs and the status bar, then the compositor waterfall from synthetic data, and finally real IQ from the PlutoSky.

C Host Tests

The shared UI and protocol code and the C rendering model have a host test suite that runs without any hardware attached. Tests cover the UI state struct packing and defaults, wire protocol frame encode and decode including CRC16-CCITT validation, golden-image output from the pixel shader, golden-image output from the framebuffer compositor, framebuffer swap and access behavior, and the touch and UI state-machine event sequences. Golden images pin the expected visual output for each scenario: a default screen, the spectrum trace, the touch crosshair, the mute indicator, a waterfall stepped several rows, and others. A change that shifts a single pixel fails the suite without a board attached.

Verilator Parity

The pixel shader and ROM wrappers are checked against the C reference model using Verilator. The same inputs feed both the RTL shader and the C reference, and the job fails if any output byte differs. The generated font and sprite memory files are compared against their C source arrays the same way. Because the C model is also the gateware specification, these checks confirm the RTL produces pixel-identical output to the reference.

AI Usage

We used AI as a coding assistant on well-scoped parts of the project, and it helped most where we could check its output exactly.

- **For the display path,** we used AI to write SystemVerilog modules and the C reference model they are checked against. The two are tied to the same specification by an end-to-end harness: a Verilator job feeds identical inputs through the RTL and the C model and fails if a single pixel comes out different, and the generated ROMs are compared the same way. Because of that, anything AI wrote for the display had to clear a hard test before we kept it, namely matching the reference pixel for pixel. The harness did the judging in place of a code review, which is what made it worth leaning on AI here.
- **For the test suite,** we used AI to fill out coverage, turning a described scenario into a golden-image or unit test, which we then checked once against the live harness and locked in as a regression.